

**** 9.20** 编写你自己的 malloc 和 free 版本，将它的运行时间和空间利用率与标准 C 库提供的 malloc 版本进行比较。

练习答案

9.1 这道题让你对不同地址空间的大小有了些了解。曾几何时，一个 32 位地址空间看上去似乎是无法想象的大。但是，现在有些数据库和科学应用需要更大的地址空间，而且你会发现这种趋势会继续。在有生之年，你可能会抱怨个人电脑上那狭促的 64 位地址空间！

虚拟地址位数 (n)	虚拟地址数 (N)	最大可能的虚拟地址
8	$2^8 = 256$	$2^8 - 1 = 255$
16	$2^{16} = 64K$	$2^{16} - 1 = 64K - 1$
32	$2^{32} = 4G$	$2^{32} - 1 = 4G - 1$
48	$2^{48} = 256T$	$2^{48} - 1 = 256T - 1$
64	$2^{64} = 16\ 384P$	$2^{64} - 1 = 16\ 384P - 1$

9.2 因为每个虚拟页面是 $P=2^p$ 字节，所以在系统中总共有 $2^n/2^p=2^{n-p}$ 个可能的页面，其中每个都需要一个页表条目 (PTE)。

n	$P=2^p$	PTE 的数量
16	4K	16
16	8K	8
32	4K	1M
32	8K	512K

9.3 为了完全掌握地址翻译，你需要很好地理解这类问题。下面是如何解决第一个子问题：我们有 $n=32$ 个虚拟地址位和 $m=24$ 个物理地址位。页面大小是 $P=1KB$ ，这意味着对于 VPO 和 PPO，我们都需要 $\log_2(1K)=10$ 位。(回想一下，VPO 和 PPO 是相同的。)剩下的地址位分别是 VPN 和 PPN。

P	VPN 位数	VPO 位数	PPN 位数	PPO 位数
1KB	22	10	14	10
2KB	21	11	13	11
4KB	20	12	12	12
8KB	19	13	11	13

9.4 做一些这样的手工模拟，能很好地巩固你对地址翻译的理解。你会发现写出地址中的所有位，然后在不同的位字段上画出方框，例如 VPN、TLBI 等，这会很有帮助。在这个特殊的练习中，没有任何类型的不命中：TLB 有一份 PTE 的副本，而缓存有一份所请求数据字的副本。对于命中和不命中的一些不同的组合，请参见习题 9.11、9.12 和 9.13。

A. 00 0011 1101 0111

参数	值
VPN	0xf
TLB 索引	0x3
TLB 标记	0x3
TLB 命中？ (是/否)	是
缺页？ (是/否)	否
PPN	0xd

C. 0011 0101 0111

D.

参数	值
CO	0x3
CI	0x5
CT	0xd
高速缓存命中? (是/否)	是
高速缓存字节返回	0x1d

9.5 解决这个问题将帮助你很好地理解内存映射。请自己独立完成这道题。我们没有讨论 open、fstat 或者 write 函数，所以你需要阅读它们的帮助页来看看它们是如何工作的。

code/vm/mmapcopy.c

```
1  #include "csapp.h"
2
3  /*
4   * mmapcopy - uses mmap to copy file fd to stdout
5   */
6  void mmapcopy(int fd, int size)
7  {
8      char *bufp; /* ptr to memory-mapped VM area */
9
10     bufp = Mmap(NULL, size, PROT_READ, MAP_PRIVATE, fd, 0);
11     Write(1, bufp, size);
12     return;
13 }
14
15 /* mmapcopy driver */
16 int main(int argc, char **argv)
17 {
18     struct stat stat;
19     int fd;
20
21     /* Check for required command-line argument */
22     if (argc != 2) {
23         printf("usage: %s <filename>\n", argv[0]);
24         exit(0);
25     }
26
27     /* Copy the input argument to stdout */
28     fd = Open(argv[1], O_RDONLY, 0);
29     fstat(fd, &stat);
30     mmapcopy(fd, stat.st_size);
31     exit(0);
32 }
```

code/vm/mmapcopy.c

9.6 这道题触及了一些核心的概念，例如对齐要求、最小块大小以及头部编码。确定块大小的一般方法是，将所请求的有效载荷和头部大小的和舍入到对齐要求（在此例中是 8 字节）最近的整数倍。比如，malloc(1)请求的块大小是 4+1=5，然后舍入到 8。而 malloc(13)请求的块大小是 13+4=17，舍入到 24。

请求	块大小(十进制字节)	块头部(十六进制)
malloc(1)	8	0x9
malloc(5)	16	0x11
malloc(12)	16	0x11
malloc(13)	24	0x19

9.7 最小块大小对内部碎片有显著的影响。因此，理解和不同分配器设计和对齐要求相关联的最小块大小是很好的。很有技巧的一部分是，要意识到相同的块可以在不同时刻被分配或者被释放。因此，最小块大小就是最小已分配块大小和最小空闲块大小两者的最大值。例如，在最后一个子问题中，

最小的已分配块大小是一个4字节头部和一个1字节有效载荷，舍入到8字节。而最小空闲块的大小是一个4字节的头部和一个4字节的脚部，加起来是8字节，已经是8的倍数，就不需要再舍入了。所以，这个分配器的最小块大小就是8字节。

对齐要求	已分配块	空闲块	最小块大小(字节)
单字	头部和脚部	头部和脚部	12
单字	头部, 但是没有脚部	头部和脚部	8
双字	头部和脚部	头部和脚部	16
双字	头部, 但是没有脚部	头部和脚部	8

- 9.8 这里没有特别的技巧。但是解答此题要求你理解简单的隐式链表分配器的剩余部分是如何工作的，是如何操作和遍历块的。

code/vm/malloc/mm.c

```

1 static void *find_fit(size_t asize)
2 {
3     /* First-fit search */
4     void *bp;
5
6     for (bp = heap_listp; GET_SIZE(HDRP(bp)) > 0; bp = NEXT_BLKP(bp)) {
7         if (!GET_ALLOC(HDRP(bp)) && (asize <= GET_SIZE(HDRP(bp)))) {
8             return bp;
9         }
10    }
11    return NULL; /* No fit */
12 #endif
13 }
```

code/vm/malloc/mm.c

- 9.9 这又是一个帮助你熟悉分配器的热身练习。注意对于这个分配器，最小块大小是16字节。如果分割后剩下的块大于或者等于最小块大小，那么我们就分割这个块(第6~10行)。这里唯一有技巧的部分是要意识到在移动到下一块之前(第8行)，你必须放置新的已分配块(第6行和第7行)。

code/vm/malloc/mm.c

```

1 static void place(void *bp, size_t asize)
2 {
3     size_t csize = GET_SIZE(HDRP(bp));
4
5     if ((csize - asize) >= (2*DSIZE)) {
6         PUT(HDRP(bp), PACK(asize, 1));
7         PUT(FTRP(bp), PACK(asize, 1));
8         bp = NEXT_BLKP(bp);
9         PUT(HDRP(bp), PACK(csize-asize, 0));
10        PUT(FTRP(bp), PACK(csize-asize, 0));
11    }
12    else {
13        PUT(HDRP(bp), PACK(csize, 1));
14        PUT(FTRP(bp), PACK(csize, 1));
15    }
16 }
```

code/vm/malloc/mm.c

- 9.10 这里有一个会引起外部碎片的模式：应用对第一个大小类做大量的分配和释放请求，然后对第二个大小类做大量的分配和释放请求，接下来是对第三个大小类做大量的分配和释放请求，以此类推。对于每个大小类，分配器都创建了许多不会被回收的存储器，因为分配器不会合并，也因为应用不会向这个大小类再次请求块了。